

Empirical study of deep reinforcement learning algorithms for CartPole problem

Yogesh Kumar
computer science Engineering
UPES
Dehradun, India
yogeshchauhan114@gmail.com

Pankaj Kumar
computer science Engineering
Nirma University
Ahmedabad, India
pankaj.kumar@nirmauni.ac.in

Abstract—CartPole problem is a classical control problem, which is important in many fields, including robotics and self-driving cars. Deep reinforcement learning is applied for controlling the CartPole balancing problem. This study compares the performance of four deep reinforcement learning algorithms: Deep Q-Network (DQN), Proximal Policy Optimization (PPO), QR (Quantile Regression) DQN, and Advantage Actor-Critic (A2C). The CartPole balance problem in the OpenAIGym environment is taken into consideration to execute the deep reinforcement learning algorithms. Many deep Reinforcement algorithms exist, and selecting the right one might be difficult depending on the problem's characteristics. Our findings can help in the ranking of the four methods based on how well they fit the requirements of a given application. The performance in terms of training time, training timesteps, maximum value of episode-reward-mean, and average frames per second for each method are evaluated. The results demonstrate that the best approach for resolving this problem is to use PPO method with a set of prespecified hyperparameters.

Index Terms—Proximal Policy Optimization, Advantage Actor-Critic, Deep Q-Network, QR (Quantile Regression) DQN, Stable Baselines3, CartPole, Classical Control Problem

I. INTRODUCTION

Reinforcement learning (RL) is a part of machine learning, which is motivated by behaviourist psychology of animal and human behavior. By using exploration and exploitation techniques, the RL agent take actions in the environment and learns how to behave in it. In exchange, the environment rewards the agent. [1]. In contrast to supervised learning, where the data is given, an agent in an RL framework gathers experiences by interacting with its environment. To maximise the cumulative rewards, this experience is collected. One of the most fascinating subfields of artificial intelligence nowadays is Deep Reinforcement Learning (DRL), which combines Deep Learning (DL) and RL. DRL has the ability to use human-like intelligence to tackle a variety of complicated real-world decision-making problems. DRL was introduced to deal with the sequential decision making problems that arise in numerous applications such as industrial control, video games, communication, driverless vehicle, finance, medical, and robotics [2]–[8].

The CartPole balancing problem is a common problem used for control a system. This system's dynamics are utilised

Identify applicable funding agency here. If none, delete this.

to understand tasks like maintenance of balance (e.g. self-balancing robot). The swing-up and stability of a pole can be achieved using a variety of control design strategies. Example include PID controllers, linear quadratic regulators, energy-based controllers, and fuzzy logic controllers [9]–[11]. Significant advancements have been made in control engineering [12].

A rise in system complexity necessitates the use of sophisticated controllers, particularly when nonlinearities, uncertainty, and time fluctuations are present. According to its nature, RL has the potential to leverage environmental knowledge during training to generate optimal controllers without having any prior knowledge of the environment. Additionally, these controllers have the potential to adapt the changing environment. DRL techniques provide a way to solve optimal control problems [13]. For building a control system, RL can reduce the required amount of programming time. This can be accomplished by considering the controller of CartPole problem as a RL agent. DRL techniques have been proven to be intelligent alternatives to classical control techniques [14]. In the literature, significant progress has been done using DRL in the field of classical control systems and continuous control domain [15]–[18].

Finding the ideal policy or choosing the ideal actions—is a difficult challenge when dealing with real-world RL situations with enormous or infinite action spaces. Even though there are many algorithms for these kinds of problem, there is still uncertainty and ambiguity about how well each approach performs in comparison to the others. Specifically, choosing the method that works best for a task is difficult. Therefore, the purpose of this paper is to construct a comparison of Deep Q-Networks (DQN) [19], Advantage Actor-Critic (A2C) [20], Proximal Policy Optimization (PPO) [21] and QR (Quantile Regression) DQN [22] to identify which algorithm gives high performance in the form of training time, episode-reward-mean and average reward for the discrete action spaces systems.

II. CARTPOLE PROBLEM

The CartPole balancing problem [23] is the case study for comparison. It is a well known reinforcement problem in classical control environment. In CartPole problem a pole

is mounted on a hinge joint on a cart. It is very similar to inverted pendulum problem. The system is unstable by design and is balanced by moving the pivot point vertically below the center of mass. Very similar to the problem of balancing a bamboo pole on human palm. The pole can freely move only in the vertical plane of the cart. We restrict the degree of freedom to one dimension only when compared to human sport of balancing a pole on human palm. In order to balance the pole, forces in the left and right directions are applied to the cart, until the pole is balanced upright.

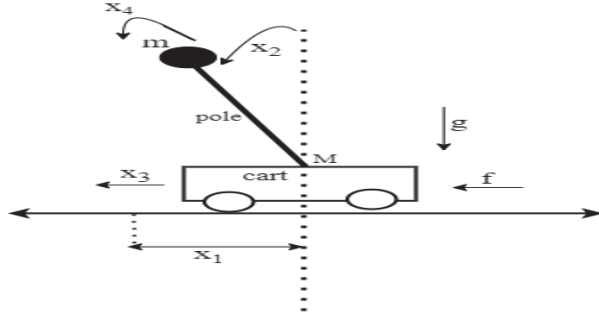


Fig. 1: CartPole balancing system where $(\vec{x}_1, x_2, \vec{x}_3, \vec{x}_4)$ is the cart's position, pole's angle, cart's velocity, and pole's angular velocity. M signify cart's mass and m is the pole's mass; g and f represent the gravitational acceleration and external force.

The environment and graphical interface can be generated with the Open-AI simulator [24]. Figure 1. is a schematic of the CartPole balancing environment. The observation space is defined by the four real values $\{\vec{x}_1, x_2, \vec{x}_3, \vec{x}_4\}$ cart's position on the track, pole's angle with the vertical, cart's velocity and pole's angular velocity. M and m signifies the cart's mass and pole's mass; g and f represent the gravitational acceleration and external force. The velocity change with the applied force, which is not fixed and depends on the angle at which the pole is pointing. In our simulation these are the design aspects: $(-4.8, 4.8)$ are the range of cart position. If the cart position moves greater than the range of $(-2.4, 2.4)$, the episode ends. The range of the pole angle lies between $(-24^\circ, 24^\circ)$. If the angle of pole exceeds the range of $(-12^\circ, 12^\circ)$, the episode ends. The pole must remain upright for as long as feasible, thus each step taken including the ending step receives a reward of +1. In CartPole-v1 system, the threshold value of the rewards for each episode is 475.

A. Markov Decision Processes

Markov Decision Processes (MDP) [25] is used to create the mathematical framework for RL problem. Markov Decision Process (MDP), which is an extension of the Markov process. The Markov process can be represented mathematically as a discrete-time stochastic process that mainly depends on the current state to determine the probability distribution of potential future states. It offers a framework for simulating

decision-making processes. Here MDP is represented by the five tuples $X, A, P, R,$ and γ where

- X : observation space $\{\vec{x}_1, x_2, \vec{x}_3, \vec{x}_4\}$
- A : action space (0: push cart to the left and 1: push cart to the right).
- P : $X \times A \times X \rightarrow [0,1]$ This represents transition probability from one state x_t to the next x_{t+1} by taking already defined action.
- R : reward function $R: X \times A \rightarrow R$ where the agent gets reward r after taking an action while changing state x_t to the x_{t+1} . In this case if the pole stays upright, the reward +1 is awarded at each timestep.
- γ discount factor which contains the value between 0 and 1. Depending on the algorithm being used, the discount factor changes.

B. Deep Q-Network (DQN)

Deep Q-Network (DQN) [19] is one of the well-known algorithms in deep RL, which can learn policies from high dimensional inputs. In DQN inputs are the states and output is the action value function which helps to estimate the reward function. Consider the regression problem for an instance, z represents the target value of a classical control problem and x represents state of the agent, for this problem input is (x, a) , target $z(x, a)$ and the mean square error loss function is as follows:

$$L^{DQN} = L(z(x_t, a_t), Q^*(x_t, a_t, \omega_t)) = \|z(x_t, a_t) - Q^*(x_t, a_t, \omega_t)\|^2 \quad (1)$$

$$z(x_t, a_t) = R(x_t, x_{t+1}) + \gamma \max_{a_{t+1}} Q^*(x_{t+1}, a_{t+1}, \omega_t) \quad (2)$$

Where ω represents the network parameters, $\omega \in R^{|X||R|}$ and x_{t+1} is a sample taken from $T(x_{t+1}|x_t, a_t)$ with the input of (x_t, a_t) . Using gradient descent technique update the ω for minimizing the loss function.

$$\omega_{t+1} = \omega_t - \alpha_t \frac{\delta L^{DQN}}{\delta \omega} \quad (3)$$

DQN algorithm can be written as:

Algorithm 1 DQN

Set the capacity of replay memory $\mathcal{D}$ to N

The action-value function Q initialised using two arbitrary weights ω, ω'

for $episode = 1, M$ **do**

for $t = 1, T$ **do**

 With probability ε , choose a action a_t randomly

 Otherwise, choose $a_t = \arg \max_a Q(x_t, a; \omega)$

 Perform action a_t , receive reward value r_{t+1} and notice the next state x_{t+1}

 Put the transition elements $(x_t, a_t, r_{t+1}, x_{t+1})$ in replay memory $\mathcal{D}$

 Take sample batch of transitions $(x_j, a_j, r_{j+1}, x_{j+1})$ from $\mathcal{D}$

 Set $y_j = \begin{cases} r_{j+1}, & \text{if } x_{j+1} \text{ is terminal} \\ r_{j+1} + \gamma \max_{a'} Q(x_{j+1}, a'; \omega'), & \text{otherwise} \end{cases}$

 Implement a gradient descent step using targets y_j with respect to the online parameters ω on $(y_j - Q(x_t, a_t; \omega))^2$

 For each C steps, update $\omega' \leftarrow \omega$

end

end

C. Advantage Actor-Critic (A2C)

Advantage Actor-Critic [20] is a RL algorithm that contains actor and critic two neural networks, actor representing for policy $\pi_\theta(a_t|x_t)$ and critic network V_ω^π evaluate the actor performance with parameter ω . The advantage function determines: how much better that state's action taken is compared to the state's average value. It is the difference between the state action pair and the average value of that state.

$$A^\pi(x_t, a_t) = Q^\pi(x_t, a_t) - V^\pi(x_t) \quad (4)$$

Using Bellman optimization above equation can be written as:

$$A^\pi(x_t, a_t) = R(x_t, x_{t+1}) + \gamma V_\omega^\pi(x_{t+1}) - V_\omega^\pi(x_t) \quad (5)$$

Like DQN, target function can be written as:

$$y = R(x_t, x_{t+1}) + \gamma V_\omega^\pi(x_{t+1}) \quad (6)$$

A2C algorithm can be implemented as:

Algorithm 2 A2C

- Use Eq.4. to calculate the advantage function.
 - Use Eq.6. to calculate the target function.
 - Calculate critic loss: $L = \frac{1}{Batchsize} \sum_T ||y - V^\pi(x_t)||^2$
 - Calculate gradient for the critic: $\nabla^{critic} = \frac{\delta L}{\delta \omega}$
 - Calculate gradient for the actor: $\nabla^{actor} = \frac{1}{Batchsize} \sum_T \nabla_\theta \log_\pi(a_t|x_t) A^\pi(x_t, a_t)$
-

D. Proximal Policy Optimization (PPO)

The Proximal Policy Optimisation (PPO) [21] increases the training stability of our agent by preventing excessive policy updates. To achieve this, we clip a ratio from a predetermined range $[1 - \epsilon, 1 + \epsilon]$ that will show the difference between our

present and previous policies. The objective function for the PPO algorithm can be written as:

$$L^{CLIP}(\omega) = \hat{E}_t[\min(r_t(\omega)\hat{A}_t, \text{clip}(r_t(\omega), 1 - \epsilon, 1 + \epsilon)\hat{A}_t)] \quad (7)$$

Here, $r_t(\omega)$ is the ratio function which is calculated as follows:

$$r_t(\omega) = \frac{\pi_\omega(a_t|x_t)}{\pi_{\omega_{old}}(a_t|x_t)}$$

$r_t(\omega)$ represents the probability ratio between present and old policy. This probability ratio is a simple approach to calculating how much the current policy differs from the previous one. The final objective function for PPO in the Actor-Critic style is defined as:

$$L_t^{CLIP+VF+X}(\omega) = \hat{E}_t[L_t^{CLIP}(\omega) - c_1 L_t^{VF}(\omega) + c_2 X[\pi_\omega](x_t)] \quad (8)$$

Above function is the combination of clipped objective function $L_t^{CLIP}(\omega)$, value loss function $L_t^{VF}(\omega)$ and entropy bonus $X[\pi_\omega](x_t)$. PPO algorithm can be written as:

Algorithm 3 PPO

for $iteration=1, 2, \dots$ **do**

for $actor = 1, 2, \dots, N$ **do**

 For T timesteps, execute policy $\pi_{\omega_{old}}$ in the environment

 To find advantage estimates $\hat{A}_1, \hat{A}_2, \dots, \hat{A}_T$

end

 optimize objective function Eq.3., with K epochs and minibatch size $M \leq NT$

$\omega_{old} \leftarrow \omega$

end

E. QR (Quantile Regression) DQN

QR (Quantile Regression) DQN [22] is a value based and model free RL algorithm. The positions of the distributions are stochastically adjusted using QRDQN using quantile regression to reduce the Wasserstein distance [26] to a target distribution. For a quantile $\tau \in [0, 1]$, QR loss is a loss function which gives penalty τ when there will be overestimation errors and $1 - \tau$ for the underestimation error. If given distribution is Z , and quantile τ , then the quantile function value $F_Z^{-1}(\tau)$ might be characterized as the quantile regression loss minimizer:

$$\mathcal{L}_{QR}^\tau(\omega) = E_{\hat{z} \sim Z}[\rho_\tau(\hat{z} - \omega)] \quad (9)$$

$$\rho_\tau(u) = u(\tau - \delta_{\{u < 0\}}), \forall u \in R \quad (10)$$

Above loss function is not uniform at zero, for the non-linear function approximation it may hinder performance. Due to this, while updating QRDQN using the Bellman method, a modified quantile loss known as the quantile huber [27] loss is used.

$$\rho^k_\tau(u) = L_k(u)|\tau - \delta_{\{u < 0\}}|, \text{ where } L_k \text{ is Huber Loss} \quad (11)$$

Algorithm 4 QRDQN**Require** N, k

Input state x , action a , reward r , next_state x' , discount_factor $\gamma \in [0, 1]$
 // To find distributional bellman target
 $Q(x', a') = \sum_j q_j \omega_j(x', a')$
 $a^* \leftarrow \arg\max_{a'} Q(x, a')$
 $\tau \omega_j \leftarrow r + \lambda \omega_j(x', a^*), \forall_j$
 // To find quantile regression loss
Output $\sum_{i=1}^N E_j[\rho_{\tau_i}^k(\tau \omega_j - \omega_i(x, a))]$

1) Stable-Baselines3

Stable-Baselines3 [28] is used here to compare the Deep Q-Network, A2C, Proximal Policy Optimisation, and QRDQN. Stable Baselines3 (SB3) is a collection of trustworthy PyTorch implementations of reinforcement learning techniques. There are many RL algorithms available in stable baselines3. In which different algorithms perform better for different environments. This is the next updated version of the stable baseline. By establishing solid foundations on which to base initiatives, these algorithms will make it simpler for the scientific community and industry to reproduce, improve, and find new ideas.

III. RESEARCH METHODOLOGY

In this work we used process pipeline:

- Choosing the algorithm,
- Data preparation,
- Training,
- Model Evaluation and testing,
- Hyperparameter tuning.

DRL algorithms are based more on the action space not so much on observation space. Certain algorithms can only work on certain types of action spaces, like DQN works on “Discrete” spaces only and DDPG [15] works on “Box” spaces only. So according our problem we need to select those algorithm which can work on discrete action spaces. DQN, A2C, PPO and QRDQN algorithm works with discrete action spaces. In **Data preparation** phase we define the training phase’s settings like training timesteps, hyperparameters, reward, action and observation spaces. We have decided number of timesteps and the value of hyperparameters according the behaviour of algorithms. MDP for CartPole problem already defined in gym library, where observation space, action space, reward, starting state and terminate condition clearly defined.

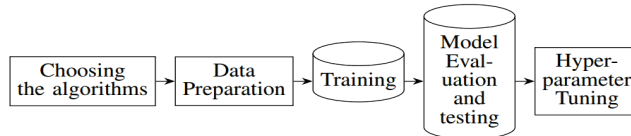


Fig. 2: Process pipeline for choosing algorithms, data preparation, training, and model evaluation and testing steps as used in this work.

In **Training** phase agent learn how to balance the pole using DRL algorithms. In CartPole problem the agent is a cart that can move left or right and is managed by two possible action values 0 (push cart to the left) and 1 (push cart to the right). In **Model evaluation and Testing** phase evaluated the models performance of the four DRL methods on 20 episodes. Finally all models tested on classical control system “CartPole” using gym environment for 5 episodes. In **hyperparameter tuning** stage, based on the findings of the evaluation, we refine trained model by adjusting the hyperparameters. Figure 2. shows the methodology used in this paper.

IV. EXPERIMENTAL RESULTS

The experimentation process provided several graphs and training metrics, which are illustrating the behavior of each algorithm. For each episode (with a maximum of 500 steps or frames) threshold value for reward is 475. The episode comes to an end if the cart moves more than 2.4 units from the centre or the pole deviates more than 12 degrees from vertical. For balancing pole we used DRL methods DQN, PPO, A2C, and QRDQN using the baselines3 library.

TABLE I: Performance evaluation of PPO, A2C, DQN and QRDQN in terms of training time, training timesteps, maximum value of episode-reward-mean during training, average frames per second after the 10 episodes during evaluation.

Technique	Training time (sec)	Training timesteps	Maximum episodes-reward-mean	Average frames per second
PPO	97	79k	500	834.9487
A2C	241	100K	488	411.515
DQN	650	120K	380	262.0905
QRDQN	353	100K	407	385.2622

In reinforcement learning algorithms, two distinct policy designs called CnnPolicy and Multi-Layer perceptron policy (MlpPolicy) are commonly used. Depending on the context and the type of input data, CnnPolicy or MlpPolicy are chosen. The CnnPolicy is typically more suited for tasks requiring image input data, whereas the MlpPolicy is better suited for tasks involving non-image input data. In this work MlpPolicy is common for all the algorithms. Table I. represents the performance comparison of PPO, A2C, DQN, and QRDQN, where training time represents the how much time taken by the algorithm to train the model. In this work PPO has taken minimum time 97 seconds to train the model. Timesteps during training refer to how many times the RL agent interacts with the environment and performs an action, receives a reward, and then changes the state. Episode-reward-mean represents the average reward that the agent accumulated per episode. DQN has taken maximum time steps 120k, during this maximum episode-reward-mean reached is 380 by using the value of learning rate is 0.00011. PPO has taken minimum time steps 79k, during this maximum episode-reward-mean reached to 500 by using the value of learning rate 0.00029 and discount factor 0.98. We create trained models using four DRL methods

such as PPO, A2C, DQN, and QRDQN using hyperparameters given in table II, III, IV, and V. for the CartPole-v1 problem. After the evaluation of models for 10 episodes, Average reward reached to 500 for all methods (PPO, A2C, DQN, and QRDQN).

TABLE II: Hyperparameters of PPO

Hyperparameter	Value
Policy	MlpPolicy
Learning rate	0.00029
Number of steps	2048
Batch size	64
Number of epochs	10
Gamma	0.98
Generalized advantage estimator lambda	0.95
Clip range	0.2
Entropy coefficient	0.0
Value function coefficient	0.5
Max_grad_norm	0.5

TABLE III: Hyperparameters of A2C

Hyperparameter	Value
Policy	MlpPolicy
Learning rate	0.00068
Number of steps	5
Gamma	0.97
Generalized advantage estimator lambda	1.0
Entropy coefficient	0.0
Value function coefficient	0.5
Max_grad_norm	0.5
RMSProp_eps	1e-05
SDE_sample_freq	-1

TABLE IV: Hyperparameters of DQN

Hyperparameter	Value
Policy	MlpPolicy
Learning rate	0.00011
Batch size	64
Buffer size	100000
Learning starts	1000
Gamma	0.99
Target update interval	10
Train frequency	256
Gradient steps	128
Exploration fraction	0.16
Exploration final epsilon	0.04
Policy kwargs	dict(net_arch={256, 256})

After then, we tested these models for 5 episodes, for each episode all models performed well and achieved a 500 reward. In this work, we have taken 100k timesteps initially to train the models but DQN was not able to train the model in 100k so we increased the number of timesteps up to 120k to obtain satisfactory result. Comparatively, PPO has taken

TABLE V: Hyperparameters of QRDQN

Hyperparameter	Value
Policy	MlpPolicy
Learning rate	0.0015
Batch size	64
Buffer size	100000
Learning starts	1000
Gamma	0.99
Target update interval	10
Train frequency	256
gradient_steps	128
Exploration fraction	0.16
Exploration final epsilon	0.04
Policy_kwargs	dict(net_arch={256, 256}, n_quantiles=10)

fewer timesteps 79k to train the model and achieved maximum episode-mean-reward 500. We observed that PPO and A2C converge fast in the comparison of DQN and QRDQN.

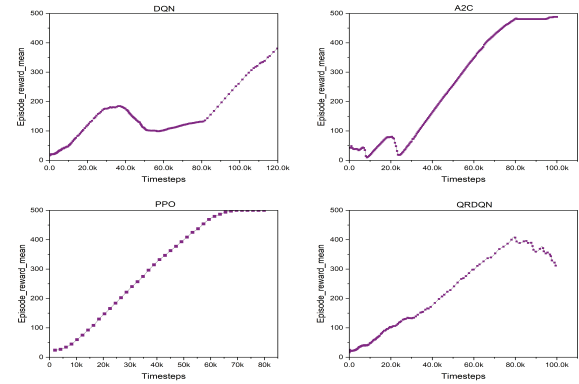


Fig. 3: Episode reward mean in PPO, A2C, DQN, and QRDQN. PPO generated the maximum episode reward mean in minimum timesteps and its graph shows monotonically increasing episode reward mean as function of timestep, where as other graphs are not so well behaved.

Figure 3. shows episode-reward-mean during training, which is the average reward that the agent collect per episode, where PPO generated the maximum episode reward mean in minimum timesteps and its graph shows monotonically increasing episode mean reward as a function of timesteps, where as other graphs are not so well behaved. From this it is known that the performance of PPO was better than others on training time. Figure 4. shows the frame per second (fps) for all the methods, where PPO achieved the highest frame processing speed comparatively. Fps stands for number of batches processed in a time period, which can vary according to the DRL algorithm, environment and its implementation. It can be calculated using the formula: $\text{fps} = \frac{n_{\text{batch}}}{t_{\text{now}} - t_{\text{start}}}$,

where $nbatch$ = number of batches, $tnow$ = current time, and $tstart$ = starting time.

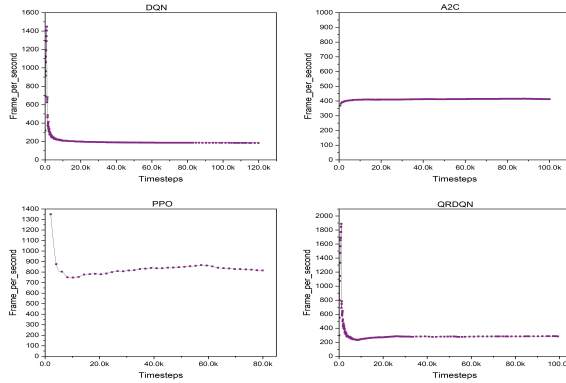


Fig. 4: Frames per second in PPO, A2C, DQN, and QRDQN. Here PPO achieved highest frames processing speed. During training, the fps stabilize for all algorithms in about 10k timesteps.

V. CONCLUSION

In this paper, state-of-the-art deep reinforcement learning algorithms are compared for the classical CartPole balancing problem. PPO, A2C, DQN, and QRD-QN used deep neural networks to represent policies. The CartPole balancing problem was implemented in the OpenAI Gym environment by using reinforcement algorithms with the help of stable baselines3. The performance of the DRL methods represents using the training time, training timesteps, episode reward mean, and average frames per second. Comparative results show that the PPO performed comparatively well for the classical control CartPole balancing problem. It was found that PPO converged fast and used less number of timesteps. From this, we can conclude that with the ability of the PPO to make policy changes, this algorithm will perform well for systems that work on discrete action spaces. During the experiment, it was observed that the logic of policy change is important for the DRL method's performance. Future work will concentrate on evaluating the effectiveness of deep reinforcement learning techniques for challenging problems, which can work on discrete action spaces with multi agent environments.

REFERENCES

- [1] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, 2018.
- [2] Z. Zhao, W. He, C. Mu, T. Zou, K.-S. Hong, and H.-X. Li, "Reinforcement learning control for a 2-dof helicopter with state constraints: Theory and experiments," *IEEE Transactions on Automation Science and Engineering*, 2022.
- [3] K. Muelling, A. Boularias, B. Mohler, B. Schölkopf, and J. Peters, "Learning strategies in table tennis using inverse reinforcement learning," *Biological cybernetics*, vol. 108, pp. 603–619, 2014.
- [4] N. C. Luong, D. T. Hoang, S. Gong, D. Niyato, P. Wang, Y.-C. Liang, and D. I. Kim, "Applications of deep reinforcement learning in communications and networking: A survey," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 4, pp. 3133–3174, 2019.
- [5] Y. Sun, Y. Chu, T. Xu, J. Li, and X. Ji, "Inverse reinforcement learning based: Segmented lane-change trajectory planning with consideration of interactive driving intention," *IEEE Transactions on Vehicular Technology*, vol. 71, no. 11, pp. 11 395–11 407, 2022.
- [6] P. N. Kolm and G. Ritter, "Modern perspectives on reinforcement learning in finance," *Modern Perspectives on Reinforcement Learning in Finance* (September 6, 2019). *The Journal of Machine Learning in Finance*, vol. 1, no. 1, 2020.
- [7] S. K. Zhou, H. N. Le, K. Luu, H. V. Nguyen, and N. Ayache, "Deep reinforcement learning in medical imaging: A literature review," *Medical image analysis*, vol. 73, p. 102193, 2021.
- [8] J. Kober, J. A. Bagnell, and J. Peters, "Reinforcement learning in robotics: A survey," *The International Journal of Robotics Research*, vol. 32, no. 11, pp. 1238–1274, 2013.
- [9] N. Muskinja and B. Tovornik, "Swinging up and stabilization of a real inverted pendulum," *IEEE transactions on industrial electronics*, vol. 53, no. 2, pp. 631–639, 2006.
- [10] Q. Shi, H.-K. Lam, C. Xuan, and M. Chen, "Adaptive neuro-fuzzy pid controller based on twin delayed deep deterministic policy gradient algorithm," *Neurocomputing*, vol. 402, pp. 183–194, 2020.
- [11] Q. Shi, H.-K. Lam, B. Xiao, and S.-H. Tsai, "Adaptive pid controller based on q-learning algorithm," *CAAI Transactions on Intelligence Technology*, vol. 3, no. 4, pp. 235–244, 2018.
- [12] O. Katsuhiko, *Modern control engineering*. Editorial Félix Varela, 2009.
- [13] Y. Li, "Deep reinforcement learning: An overview," *arXiv preprint arXiv:1701.07274*, 2017.
- [14] A. E. Bryson, *Applied optimal control: optimization, estimation and control*. Routledge, 2018.
- [15] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," *arXiv preprint arXiv:1509.02971*, 2015.
- [16] Y. Duan, X. Chen, R. Houthoofd, J. Schulman, and P. Abbeel, "Benchmarking deep reinforcement learning for continuous control," in *International conference on machine learning*. PMLR, 2016, pp. 1329–1338.
- [17] A.-I. Patachi and F. Leon, "A comparative performance study of reinforcement learning algorithms for a continuous space problem," in *2019 23rd International Conference on System Theory, Control and Computing (ICSTCC)*. IEEE, 2019, pp. 860–865.
- [18] R. Özalp, N. K. Varol, B. Taşci, and A. Uçar, "A review of deep reinforcement learning algorithms and comparative results on inverted pendulum system," *Machine Learning Paradigms: Advances in Deep Learning-based Technological Applications*, pp. 237–256, 2020.
- [19] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski et al., "Human-level control through deep reinforcement learning," *nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [20] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, "Asynchronous methods for deep reinforcement learning," in *International conference on machine learning*. PMLR, 2016, pp. 1928–1937.
- [21] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.
- [22] W. Dabney, M. Rowland, M. Bellemare, and R. Munos, "Distributional reinforcement learning with quantile regression," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018.
- [23] A. G. Barto, R. S. Sutton, and C. W. Anderson, "Neuronlike adaptive elements that can solve difficult learning control problems," *IEEE transactions on systems, man, and cybernetics*, no. 5, pp. 834–846, 1983.
- [24] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, "Openai gym," *arXiv preprint arXiv:1606.01540*, 2016.
- [25] M. L. Puterman, *Markov decision processes: discrete stochastic dynamic programming*. John Wiley & Sons, 2014.
- [26] L. N. Vaserstein, "Markov processes over denumerable products of spaces, describing large systems of automata," *Problemy Peredachi Informatsii*, vol. 5, no. 3, pp. 64–72, 1969.
- [27] P. J. Huber, "Robust estimation of a location parameter," in *Breakthroughs in statistics: Methodology and distribution*. Springer, 1992, pp. 492–518.
- [28] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dornmann, "Stable-baselines3: Reliable reinforcement learning implementations," *The Journal of Machine Learning Research*, vol. 22, no. 1, pp. 12 348–12 355, 2021.